

NLP_Practical_02

April 7, 2026

0.1 Install gensim

```
[1]: !pip install gensim
```

```
Requirement already satisfied: gensim in  
c:\users\nikki\appdata\local\programs\python\python310\lib\site-packages (4.4.0)  
Requirement already satisfied: numpy>=1.18.5 in  
c:\users\nikki\appdata\local\programs\python\python310\lib\site-packages (from  
gensim) (1.26.1)  
Requirement already satisfied: smart_open>=1.8.1 in  
c:\users\nikki\appdata\local\programs\python\python310\lib\site-packages (from  
gensim) (7.5.1)  
Requirement already satisfied: scipy>=1.7.0 in  
c:\users\nikki\appdata\local\programs\python\python310\lib\site-packages (from  
gensim) (1.14.1)  
Requirement already satisfied: wrapt in  
c:\users\nikki\appdata\local\programs\python\python310\lib\site-packages (from  
smart_open>=1.8.1->gensim) (1.17.2)
```

```
[notice] A new release of pip available: 22.2.2 -> 26.0.1
```

```
[notice] To update, run: python.exe -m pip install --upgrade pip
```

0.1.1 Step-1: Import Libraries

```
[2]: import numpy as np  
import pandas as pd  
  
import nltk  
nltk.download('punkt')  
  
from nltk.tokenize import word_tokenize  
  
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer  
  
from gensim.models import Word2Vec
```

```
[nltk_data] Downloading package punkt to
```

```
[nltk_data] C:\Users\NIKKI\AppData\Roaming\nltk_data...
```

```
[nltk_data] Package punkt is already up-to-date!
```

0.1.2 Step-2: Sample Data

```
[6]: documents = [
    "I love machine learning",
    "Machine learning is amazing",
    "I love coding in Python",
    "Python is great for data science"
]

print("Documents:\n ", documents)
```

Documents:

```
['I love machine learning', 'Machine learning is amazing', 'I love coding in
Python', 'Python is great for data science']
```

0.1.3 Step-3: Bag of Words

```
[7]: # Count Occurrence (BoW)
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(documents)

print("\n--- Bag of Words (Count Occurrence) ---")
print("Vocabulary:", vectorizer.get_feature_names_out())
print("Matrix:\n", X.toarray())
```

--- Bag of Words (Count Occurrence) ---

```
Vocabulary: ['amazing' 'coding' 'data' 'for' 'great' 'in' 'is' 'learning' 'love'
'machine' 'python' 'science']
```

Matrix:

```
[[0 0 0 0 0 0 0 1 1 1 0 0]
 [1 0 0 0 0 0 1 1 0 1 0 0]
 [0 1 0 0 0 1 0 0 1 0 1 0]
 [0 0 1 1 1 0 1 0 0 0 1 1]]
```

```
[9]: # Normalized Count
X_array = X.toarray()
normalized_X = X_array / X_array.sum(axis=1, keepdims=True)

print("\n Normalized Bag of Words ")
print(normalized_X)
```

Normalized Bag of Words

```
[[0.         0.         0.         0.         0.         0.
  0.         0.33333333 0.33333333 0.33333333 0.         0.
  0.25        0.         0.         0.         0.         0.]
]
```

```

0.25      0.25      0.      0.25      0.      0.      ]
[0.      0.25      0.      0.      0.      0.25
0.      0.      0.25      0.      0.25      0.      ]
[0.      0.      0.16666667 0.16666667 0.16666667 0.
0.16666667 0.      0.      0.      0.16666667 0.16666667]]

```

0.1.4 Step-4: TF-IDF

```

[19]: tfidf = TfidfVectorizer()
      X_tfidf = tfidf.fit_transform(documents)

      print("TF-IDF\n")
      print("Vocabulary:\n", tfidf.get_feature_names_out())
      print("Matrix:\n", X_tfidf.toarray())

```

TF-IDF

Vocabulary:

```

['amazing' 'coding' 'data' 'for' 'great' 'in' 'is' 'learning' 'love'
'machine' 'python' 'science']

```

Matrix:

```

[[0.      0.      0.      0.      0.      0.
0.      0.57735027 0.57735027 0.57735027 0.      0.      ]
[0.59081908 0.      0.      0.      0.      0.
0.46580855 0.46580855 0.      0.46580855 0.      0.      ]
[0.      0.55528266 0.      0.      0.      0.55528266
0.      0.      0.43779123 0.      0.43779123 0.      ]
[0.      0.      0.43671931 0.43671931 0.43671931 0.
0.34431452 0.      0.      0.      0.34431452 0.43671931]]

```

0.1.5 Step-3: Word2Vec

```

[20]: # Tokenize sentences
      tokenized_docs = [word_tokenize(doc.lower()) for doc in documents]

```

```

[21]: # Train Word2Vec model
      model = Word2Vec(sentences=tokenized_docs, vector_size=50, window=2,
      ↪min_count=1, workers=4)

```

```

[24]: print("Word2Vec Embeddings\n ")

      # Example: Get vector for a word
      word = "machine"
      print(f"Vector for '{word}':\n", model.wv[word])

```

Word2Vec Embeddings

Vector for 'machine':

```

[ 1.56351421e-02 -1.90203730e-02 -4.11062239e-04  6.93839323e-03

```

```

-1.87794445e-03  1.67635437e-02  1.80215668e-02  1.30730132e-02
-1.42324204e-03  1.54208085e-02 -1.70686692e-02  6.41421322e-03
-9.27599426e-03 -1.01779103e-02  7.17923651e-03  1.07406788e-02
 1.55390287e-02 -1.15330126e-02  1.48667218e-02  1.32509926e-02
-7.41960062e-03 -1.74912829e-02  1.08749345e-02  1.30195115e-02
-1.57510047e-03 -1.34197120e-02 -1.41718509e-02 -4.99412045e-03
 1.02865072e-02 -7.33047491e-03 -1.87401194e-02  7.65347946e-03
 9.76895820e-03 -1.28571270e-02  2.41711619e-03 -4.14975407e-03
 4.88066689e-05 -1.97670180e-02  5.38400887e-03 -9.50021297e-03
 2.17529293e-03 -3.15244915e-03  4.39334614e-03 -1.57631524e-02
-5.43436781e-03  5.32639725e-03  1.06933638e-02 -4.78302967e-03
-1.90201886e-02  9.01175756e-03]

```

```

[23]: # Similar words
print("\nWords similar to 'machine':")
print(model.wv.most_similar("machine"))

```

Words similar to 'machine':

```

[('amazing', 0.19010193645954132), ('data', 0.04489415884017944), ('for',
-0.01014822255820036), ('python', -0.014475265517830849), ('in',
-0.02321469783782959), ('science', -0.04406467080116272), ('coding',
-0.0948910042643547), ('great', -0.12279320508241653), ('learning',
-0.15515565872192383), ('is', -0.17424817383289337)]

```

```

[ ]:

```